

SAML Assignment 1.1 Report

Gualtiero Marenco Turi

June 7, 2025

1 Introduction

This report presents a technical account of a malware detection pipeline developed as part of a security-focused machine learning assignment. The goal was to design, train, and evaluate multiple classifiers capable of detecting malicious Android applications using the Drebin dataset. Particular emphasis was placed on preprocessing, model evaluation, robustness analysis, and adversarial evasion.

2 Dataset and Preprocessing

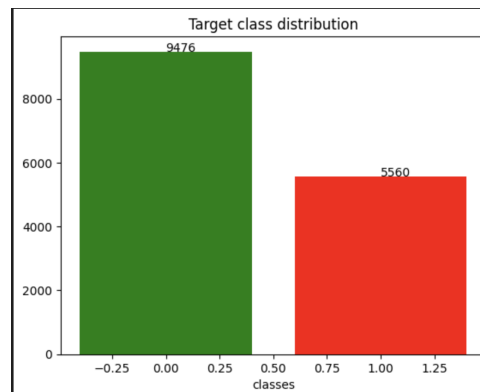
The primary dataset used contains 5560 malware samples and 9476 benign applications. Each sample is described by a set of binary features derived from various static attributes: permissions, intents, API calls, and others.

Missing values, denoted by “?” in the dataset, were handled by converting all unknowns to numeric zeros. This decision is supported by the semantic assumption that the absence of a feature indicates that the corresponding behavior or permission was not requested. This imputation strategy preserves the functional integrity of the samples and avoids introducing bias from artificial imputation.

To maintain reproducibility and support proper generalization, the dataset was split into training (70%), validation (15%), and test (15%) sets using stratified sampling to preserve class balance.

2.1 Class Imbalance and Oversampling

The class imbalance, with a benign-to-malware ratio of approximately 1.7:1, was addressed using random oversampling. Oversampling was performed strictly on the training subset to avoid information leakage and ensure that validation and test performance remain reflective of real-world class distributions.



2.2 Feature Selection via Chi-Square Test

To reduce the dimensionality of the binary feature space and increase model interpretability, a Chi-square test for independence was applied to the training set. Features with p -values below $\alpha = 0.05$ were retained, resulting in a subset of 203 statistically relevant features. This filtering step should help eliminate noise and improves classifier efficiency but excluded only very few features.

3 Model Training and Evaluation

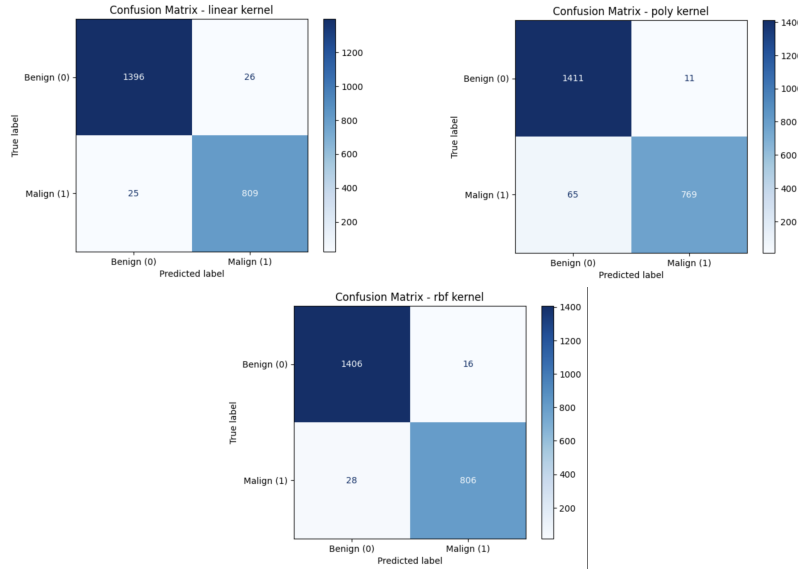
Three classifiers were selected based on diversity and typical use in malware detection: Support Vector Machines (SVM), Random Forests (RF), and Logistic Regression (LR). Evaluation was performed on the hold-out test set using confusion matrices, precision, recall, F1-score, and the Matthews Correlation Coefficient (MCC), which is robust to class imbalance and overall considered a very solid metric. False negatives are more important as the cost for misclassifying them is higher, this is not outright implemented but its formula

$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

can be tweaked to account for different weight depending on the misclassification type.

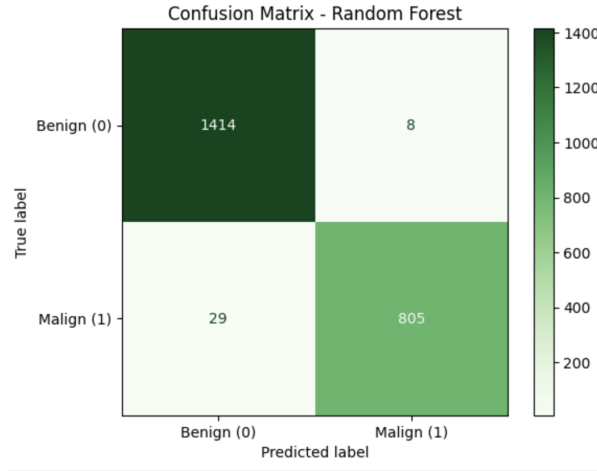
3.1 Support Vector Machines (SVM)

Three SVM kernels were tested: `linear`, `polynomial`, and `rbf`. All achieved high performance, with MCCs ranging from 0.9280 (poly) to 0.9581 (rbf). The linear kernel provided a good balance of interpretability and performance.



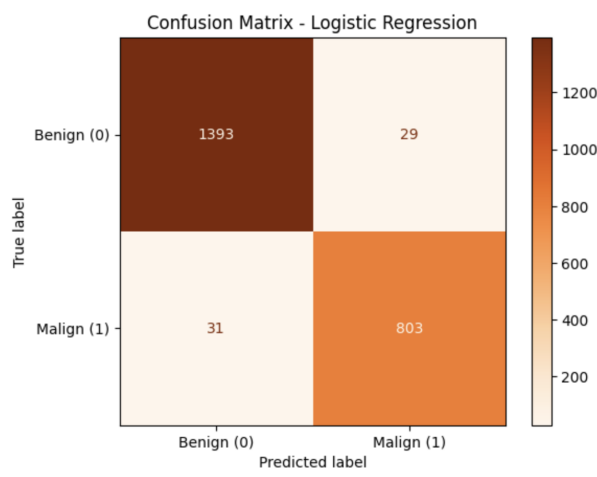
3.2 Random Forest Classifier

The Random Forest model outperformed the others with an MCC of 0.9648. Its ensemble structure allows it to handle nonlinear feature interactions and is less sensitive to outliers. The feature importance scores it provides were also crucial in the adversarial analysis.



3.3 Logistic Regression

Despite being a linear model, Logistic Regression performed admirably with an MCC of 0.9429. Its simplicity and probabilistic interpretation make it a reliable baseline. Initially I hypothesized that being a binary classification task this should have been the best solution, however it came short of RF which is notoriously well performing in classification tasks (old but gold).



4 Bypassing the Model

To assess robustness, one malware sample from the test set was selected. The objective was to transform this sample such that it would be misclassified as benign, without altering its malicious behavior. Thus, only $0 \rightarrow 1$ feature flips were allowed.

Using the Random Forest's feature importances (this part will focus on RF as it was the best performing model), the zero-valued features were ranked from least to most important. Iteratively, these low-importance features were flipped to 1 until the classifier output changed from 1 (malicious) to 0 (benign). This strategy simulates an evasion attack where a malware sample mimics benign behavior without suppressing its core functionality. I choose to start from lowest importance as it allows a more granular progression in fooling the model and, assuming more important feature are easier to discover, a stealthier approach.

The attack succeeded after flipping 88 features. This suggests that even robust classifiers like Random Forests can be vulnerable to adversarial perturbations if they rely on sparse patterns for classification. To prevent this type of attacks several approaches can be pursued,

[Insert list or table of flipped features here]

5 Discussion and Analysis

All models performed well in terms of classification accuracy and MCC. However, adversarial evaluation revealed significant differences in robustness. The Random Forest, despite its strong performance, was still susceptible to minor perturbations. Logistic Regression was harder to fool, likely because of its more conservative decision boundaries, which depend on a global linear separation.

The evaluation confirms that high test accuracy does not necessarily imply robustness. Future work should explore adversarial training, feature masking, and cost-sensitive learning to enhance the resilience of malware detectors.

6 Conclusion

A complete pipeline for static Android malware detection was constructed, evaluated, and stress-tested via adversarial manipulation. Random Forests offered the best performance, while Logistic Regression showed more robustness against evasion. These findings reinforce the importance of evaluating models not only by accuracy but also by their behavior under adaptive threats.